# Experiment 12

## 12.1.1 Phone Number Database Management

## Algorithm :

```
Step 1: Start

Step 2: Read n (number of commands)

Step 3: Initialize ph as an empty dictionary
{}

Step 4: Set i = 0

Step 5: If i < n, go to Step 6
        Else go to Step 11

Step 6: Read command (cmd)

Step 7: If cmd[0] == "ADD"
            Store ph[cmd[1]] = cmd[2]
        Go to Step 10

Step 8: Else if cmd[0] == "REMOVE"
            Remove cmd[1] from ph (if exists)
        Go to Step 10

Step 9: Else if cmd[0] == "DISPLAY"
            If ph is empty:
                Print "No contacts"
            Else:
                For each name in sorted(ph):
                    Print name and ph[name]
        Go to Step 10
```

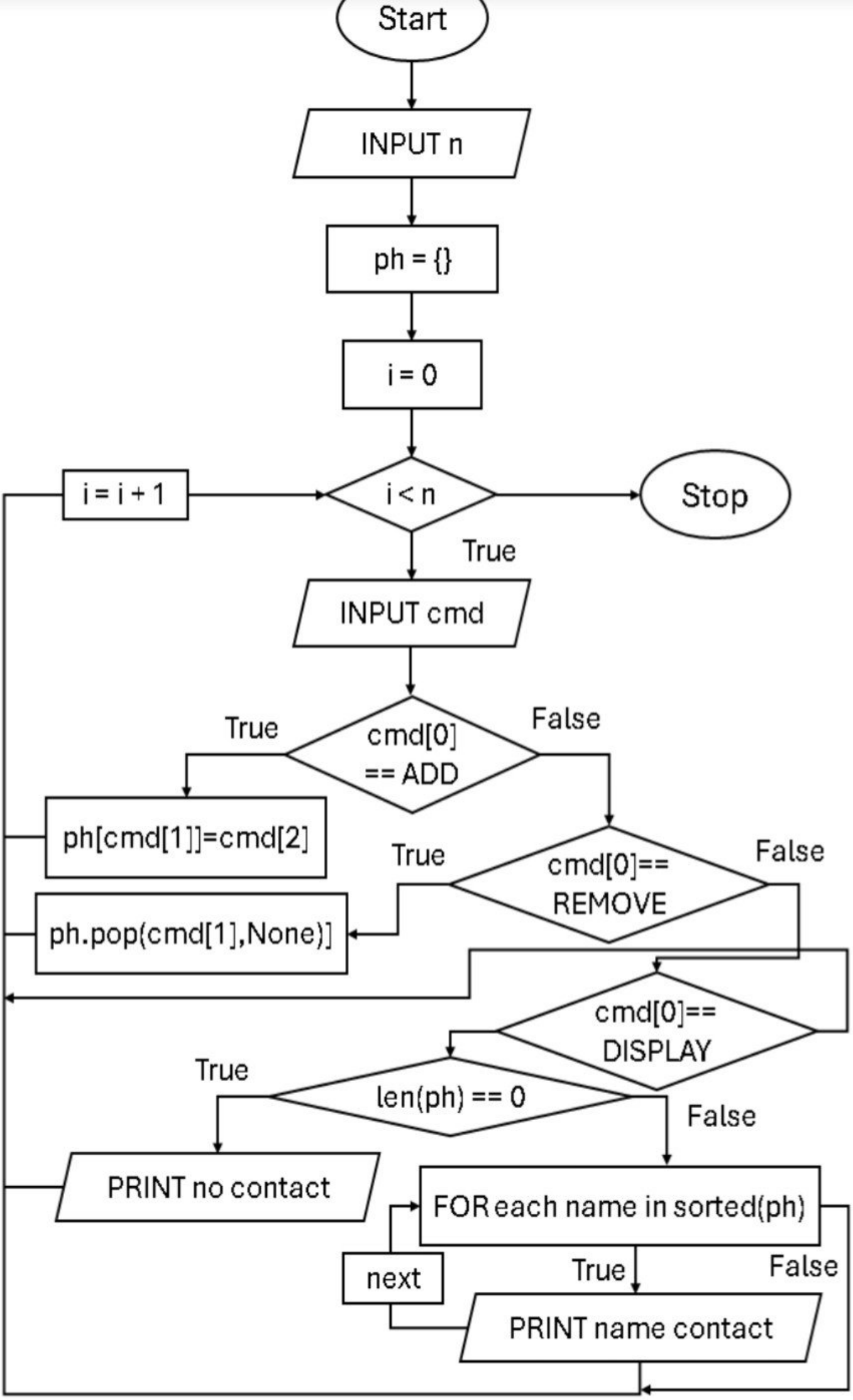
Step 10: Set i = i + 1
Go to Step 5

Step 11: Stop

## Code :

```
n=int(input())
ph={}
for i in range(n):
    cmd=input().split()
    if cmd[0] == "ADD":
        ph[cmd[1]] = cmd[2]
    elif cmd[0] == "REMOVE":
        ph.pop(cmd[1], None)
    elif cmd[0] == "DISPLAY":
        if len(ph)==0:
            print("No contacts")
        else:
            for name in sorted(ph):
                print(f"{name}: {ph[name]}")
```

## Flowchart :


Start
INPUT n
ph = {}
i = 0
i = i + 1
i < n
Stop
True
INPUT cmd
True
cmd[0]
== ADD
False
ph[cmd[1]]=cmd[2]
True
cmd[0]==
REMOVE
False
ph.pop(cmd[1],None)]
cmd[0]==
DISPLAY
True
len(ph) == 0
False
PRINT no contact
FOR each name in sorted(ph)
next
True
False
PRINT name contact

CODETANTRA Home

anwesh.lanjewar.batch2025@sitnagpur.siu.edu.in Support Logout

## 12.1.1. Phone Number Database Management

25:03

Write a Python program to manage a phone number database using a dictionary. The dictionary stores contact names as keys and phone numbers as values.

The program should support three operations: adding a contact (if the name already exists, update the phone number), removing a contact (if the name does not exist, ignore the operation), and displaying all contacts in alphabetical order by name.

**Input Format:**

1. First line contains an integer $n$ representing the number of operations.
2. Each of the next $n$ lines contains a single operation in one of the following formats:
   - "ADD <name> <phone_number>" where <name> is a string, <phone_number> is a string of 10 digits.
   - "REMOVE <name>" - name is a string.
   - "DISPLAY" - no additional parameters.

**Output Format:**

- For display operation, print all contacts in alphabetical order by name in the format:

```
<name>: <phone_number>
```

- If the database is empty during display operation, print: "No contacts".
- Do not print anything for add and remove operations.

**Constraints:**

- Name contains only alphabetic characters (no spaces). And it is case sensitive.

Sample Test Cases +

Explorer

phoneDat...

Submit

```python
n=int(input())

ph={}

for i in range(n):
    cmd=input().split()
    if cmd[0] == "ADD":
        ph[cmd[1]] = cmd[2]
    elif cmd[0] == "REMOVE":
        ph.pop(cmd[1], None)
    elif cmd[0] == "DISPLAY":
        if len(ph)==0:
            print("No contacts")
        else:
            for name in sorted(ph):
                print(f"{name}: {ph[name]}")
```

Debugger

| Average time | Maximum time |
|---|---|
| 0.011 s | 0.015 s |
| 10.50 ms | 15.00 ms |

5 out of 5 shown test case(s) passed

3 out of 3 hidden test case(s) passed

Test case 1 15 ms Debug

| Expected output | Actual output |
|---|---|
| 4 | 4 |
| ADD Alice 9876543210 | ADD Alice 9876543210 |
| ADD Bob 8765432109 | ADD Bob 8765432109 |
| ADD Charlie 7777777777 | ADD Charlie 7777777777 |
| DISPLAY | DISPLAY |
| Alice: 9876543210 | Alice: 9876543210 |

Terminal Test cases

Prev Reset Submit Next